ACTIVIDAD COLABORATIVA #2 – APLICACIÓN CLIENTE-SERVIDOR CONCURRENTE CON TCP

UNIDAD 2

UNIREMINGTON

ASTRID VIVIANA NOVA CORREA

PROFESORA: LELI LILIANA DÍAZ IZQUIERDO

2026

EXPLICACIÓN DE EL PROYECTO

SISTEMA DE CONSULTA DE INVENTARIO Y PRECIOS PARA UNA TIENDA

Propósito: Mi propósito en el proyecto era simular un entorno comercial donde múltiples clientes (por ejemplo, cajas registradoras o terminales de consulta) se conectarán simultáneamente a un servidor central de la tienda para consultar la información en tiempo real sobre la disponibilidad y el precio de los productos mediante códigos SKU.

LA ARQUITECTURA CLIENTE-SERVIDOR

- El **Servidor** actúa como el núcleo central que administra una base de datos simulada (un mapa de productos con precios) que utiliza un pool de hilos (`ExecutorService`) para atender a múltiples clientes al mismo tiempo de forma independiente y segura.

- Los **Clientes** son aplicaciones independientes que se conectan a la dirección IP y al puerto del servidor para enviar peticiones de consulta y recibir respuestas instantáneas.

FUNCIONAMIENTO DE LA COMUNICACIÓN

Quién inicia la escucha:

-El **Servidor: Lo que hace es** crear un objeto `ServerSocket` asociado a un puerto específico (`5000`) que se pone a la escucha pasiva (`accept()`), esperando activamente a que los clientes soliciten conectarse.

Quién establece la conexión:

-El **Cliente:** Inicia la conexión de forma activa instanciando un objeto `Socket` con la dirección del servidor y el puerto. El servidor acepta esta petición y le asigna un socket dedicado.

Qué respuesta produce el servidor:

-El **Servidor:** Es aquel que procesa el código recibido, lo busca en su inventario interno y responde enviando una cadena con la descripción detallada del producto, su precio y su stock actual. Ya que, si el código no existe, responde con un mensaje de error notificando que el producto no fue encontrado.

Mensajes Claros Durante la Ejecución (Ejemplo de Funcionamiento)

Mi aplicación cuenta con mensajes descriptivos en la consola que permiten rastrear en todo momento qué está ocurriendo. A continuación, se muestra cómo se visualiza la ejecución:

CONSOLA DEL SERVIDOR (AL INICIAR Y RECIBIR CLIENTES CONCURRENTES):

```
[SERVIDOR] Iniciando Sistema Central de Inventario...
==================================================
[SERVIDOR] Estado: Escuchando activamente en el puerto 5000...
[SERVIDOR] ¡Nuevo cliente conectado! Asignado ID: Cliente #1
[SERVIDOR] ¡Nuevo cliente conectado! Asignado ID: Cliente #2
[SERVIDOR] Petición recibida del [Cliente #1] -> Código: P001
[SERVIDOR] Respuesta enviada al [Cliente #1]: PRODUCTO ENCONTRADO ->
[SERVIDOR] Petición recibida del [Cliente #2] -> Código: P003
[SERVIDOR] Respuesta enviada al [Cliente #2]: PRODUCTO ENCONTRADO ->
[SERVIDOR] El [Cliente #1] solicitó finalizar la sesión.
[SERVIDOR] Socket cerrado para el [Cliente #1]. Recursos liberados.
--------------------------------------------------
```

```
-> Laptop Gamer - Precio: $1200.00 | Stock: 5 unidades

-> Mouse Inalámbrico - Precio: $25.00 | Stock: 30 unidades
```

CONSOLA DE UN CLIENTE (DURANTE SU INTERACCIÓN):

```
[CLIENTE] Intentando conectar con el servidor en localhost:5000...
[CLIENTE] ¡Conexión establecida exitosamente con el servidor!
Mensaje del Servidor: ¡Bienvenido al Servidor de Inventario! Códigos válidos: P001,

Ingrese el código del producto a consultar (P001 - P004) o 'SALIR': P001
-> Respuesta del Servidor: PRODUCTO ENCONTRADO -> Laptop Gamer - Precio: $1200.00 |

Ingrese el código del producto a consultar (P001 - P004) o 'SALIR': SALIR
-> Respuesta del Servidor: CONEXION_TERMINADA
[CLIENTE] El servidor confirmó el cierre de la conexión.
[CLIENTE] Aplicación finalizada.
```

```
, P002, P003, P004. Escribe 'SALIR' para desconectar.


| Stock: 5 unidades
```

¿UTILIZE IA?

La utilice como guía para consultar rápidamente como estructurar la lógica de concurrencia en Java utilizando un pool de hilos (`ExecutorService`), asegurando que múltiples clientes pudieran conectarse al servidor de inventario de forma simultánea y segura.